HCI — AI511 | LAB 8: IMPROVE | JAIPUR ↔ DELHI CORRIDOR | EVALUATION DATE: 7 OCT 2026

DASH — Smart Rail Journey Planner

Project Documentation, Usability Improvement Report & Viva Voce Defense Guide

PROJECT LEAD & DEVELOPER
Harshit Kulshrestha

REPOSITORY MAINTAINER
Uttam (uttamshs)

TARGET EVALUATION
Lab 8 (Full Marks: 8 / 8)

GITHUB REPOSITORY
uttamshs/Railway-tatkal...

Executive Summary & Evolution Arc

In **Lab 6**, our team engineered an initial prototype of a railway booking system tailored for the high-demand Jaipur–Delhi corridor. While technically operational, empirical testing during **Lab 7** (evaluating 11 diverse participants across quantitative tasks and qualitative think-aloud protocols, yielding an average SUS of **74.09**) uncovered profound interaction bottlenecks. Users struggled with date entry friction, cognitive overload from competing visual elements, station ambiguity across the Delhi metropolitan cluster (NDLS vs. DLI vs. DEE vs. DEC vs. SSB), opaque recommendation algorithms, multi-passenger coordination paralysis, and rigid seat preference models inherited from legacy IRCTC workflows.

For **Lab 8 ("Improve")**, we undertook a rigorous redesign. Dash evolved from a narrow booking interface into a human-centered *journey planning ecosystem*. We implemented **12 high-impact functional improvements** directly mapped to Lab 7 empirical feedback, established production-grade GitHub version control with clean release hygiene, and authored end-to-end documentation. This report presents our empirical validation, software architecture, GitHub workflow, and an exhaustive viva defense framework designed to achieve full marks across all evaluation criteria.

Lab 8 Official Evaluation Plan & Score Alignment:

1. Improvement & Build (4 Marks): Systematic implementation of solutions for every limitation identified in Lab 7.

2. GitHub & Docs (2 Marks): Active version control, public GitHub repository, and rigorous README documentation.

3. Viva Defense (2 Marks): Articulate, evidence-backed answers covering Lab 6, Lab 7 testing, and Lab 8 improvements.

PART 1 Improvement & Build (4 Marks)

To satisfy the 4-mark criteria for "Implementation of meaningful improvements identified in Lab 7", our team did not merely polish cosmetic styles or arbitrarily inject extraneous features. Every single code modification in Dash is empirically grounded in participant logs, timing observations, and user frustration points documented during Lab 7.

HCI AI511 | Lab 8: Improve Documentation — DASH Project
Page 1 of 10

1.1 Empirical Usability Evidence from Lab 7

During Lab 7, we evaluated 11 participants (P01 to P11) representing diverse experience levels (occasional travellers, frequent commuters, and first-time digital booking users). The benchmark task completion was tracked alongside qualitative friction:

74.09	100%	100%	11 / 11
AVERAGE SUS SCORE	TASK 1 COMPLETION	TASK 2 COMPLETION	LOGGED USABILITY SESSIONS

User	Profile	SUS Score	Observed Friction / Participant Direct Quote	Redesign Directive Implemented in Lab 8
P01	Occasional	72.5	*"Date selection is better with a dates dropdown instead of typing."*	Replaced free-text date input with a 14-day interactive chip strip & dropdown.
P02	Occasional	75.0	*"There are too many things competing for my attention."*	Implemented Progressive Disclosure. Primary journey is linear; secondary tools nested.
P03	Frequent	85.0	*"I don't know which Delhi station I should select. Clarify what's relevant."*	Engineered Delhi Station Cluster Intelligence (NDLS, DLI, DEE, DEC, SSB) with rationale.
P04	Occasional	75.0	*"Name of this... Goibibo feels clearer. Brand identity confusing."*	Rebranded interface as **Dash**; refined visual hierarchy, logo, and metadata.
P05	Occasional	70.0	*"The recommendation tells me what is best, but I don't understand the trade-off."*	Created **Explainable Smart Match 2.0** with transparent pros/cons & priority modes.
P06	New User	70.0	*"I can't tell whether availability is real. If dummy, mention it; else show live."*	Built live railway data adapter with explicit timestamps and honest "DEMO" fallback labels.
P07	Occasional	77.5	*"The group planner doesn't really feel intelligent enough yet. Improve its algo."*	Engineered **Group Constraint Engine** (senior lower-berth, child-with-adult, coach split).
P08	Occasional	67.5	*"Fare doesn't feel complete for a group. Add clear price breakdown."*	Introduced multi-passenger group fare summary, meal add-ons, and zero hidden costs.
P09	New User	60.0	*"What happens if my preferred train becomes unavailable suddenly?"*	Added automated **Fallback/Recovery Journeys** and journey-change alerts.
P10	Occasional	87.5	*"Is it actually booking a real ticket? Why does it end at payment handout?"*	Introduced explicit "Simulated Secure Payment Handoff" to maintain ethical boundaries.
P11	Occasional	75.0	*"Light theme could be better. High contrast needed in bright light."*	Engineered instant Theme Switcher (Dark / Light mode) with WCAG AA compliance.

HCI AI511 | Lab 8: Improve Documentation — DASH Project
Page 2 of 10

1.2 Comprehensive Inventory of the 12 Lab 8 Redesign Improvements

Below is the detailed technical and interaction breakdown of each improvement implemented in the updated build:

1. Controlled Journey-Date Selector IMPLEMENTED

Lab 7 Finding: Free-text date inputs forced mental conversions of date formats (DD/MM vs MM/DD) and triggered formatting validation errors (noted by P01).
Lab 8 Implementation: A 14-day dynamic date strip featuring quick-selection chips paired with a formatted dropdown selector. Prevents keyboard syntax errors completely.
HCI Value: *Error Prevention (Nielsen #5)* and *Recognition over Recall (Nielsen #6)*.

> **Eliminates date typing errors; immediately recalculates live train schedules.**

2. Cognitive Load Reduction & Hierarchy IMPLEMENTED

Lab 7 Finding: P02 reported visual overwhelm: filters, station guides, comparisons, and passenger lists competed on one viewport simultaneously.
Lab 8 Implementation: Built a disciplined 4-step progressive disclosure flow (`Journey` → `Select Train` → `Passengers` → `Review`). Advanced modals (Station Guide, Trade-off Matrix) only reveal on explicit user demand.
HCI Value: *Aesthetic & Minimalist Design (Nielsen #8)*.

> **Primary journey dominates; secondary tools tuck into focused slide-outs.**

3. Delhi Station Cluster Intelligence IMPLEMENTED

Lab 7 Finding: Travellers to the capital rarely know whether to choose NDLS, DLI, DEE, DEC, or SSB, causing decision paralysis (raised by P03).
Lab 8 Implementation: Users can select "Delhi — All Stations". Dash treats Delhi as an integrated hub cluster, aggregates incoming services, and explains why a specific terminal is recommended based on arrival times, metro connectivity, and destination proximity.
HCI Value: *Match between System & Real World (Nielsen #2)*.

> **Cluster aggregation across 5 terminals with contextual routing justifications.**

4. Explainable Smart Match 2.0 IMPLEMENTED

Lab 7 Finding: P05 noted that labelling a train as "Recommended" felt like an opaque black-box algorithm without transparent trade-offs.
Lab 8 Implementation: Smart Match 2.0 transparently articulates pros and trade-offs ("Faster runtime, but leaves at 02:50 AM" vs "Higher fare, but confirmed 2A availability"). Users can steer priority modes: *Balanced, Best Chance, Lower Fare, Earlier Arrival, Comfort*.
HCI Value: *User Control and Freedom (Nielsen #3)* and *Transparency*.

> **Eliminates opaque "AI" claims in favor of explainable, controllable heuristic ranking.**

5. Live Railway Data & Honest Fallback IMPLEMENTED

Lab 7 Finding: P06 questioned whether availability figures were authentic or fabricated, creating trust breakdown.
Lab 8 Implementation: Server-side API integration (calling Indian Railway API and RailRadar). When upstream APIs respond, Dash tags records as `LIVE` with refresh timestamps. If external services fail, Dash explicitly displays `DEMO FALLBACK` . No fabricated seat counts are ever masked as live.
HCI Value: *Visibility of System Status (Nielsen #1)* and *Ethical Design*.

> **Server-side proxy routes with cryptographic freshness timestamps and fallback honesty.**

6. Intelligent Group Constraint Engine IMPLEMENTED

Lab 7 Finding: P07 criticized the lack of algorithmic intelligence when planning for groups with diverse age brackets.
Lab 8 Implementation: Automatic passenger demographic detection: triggers lower-berth priority for seniors (age 60+), pairs children (<12) with accompanying adults, and provides checkboxes for "Keep group together", "Avoid coach split", and "Minimize fare".
HCI Value: *Flexibility & Efficiency of Use (Nielsen #7)*.

> **Automatic constraint detection based on passenger age, relationship, and mobility.**

HCI AI511 | Lab 8: Improve Documentation — DASH Project
Page 3 of 10

7. In-Flow Preferences & Per-Leg Overrides

IMPLEMENTED

Major IRCTC Pain Point Solved: In IRCTC, changing passenger preferences or booking connecting legs requires navigating back to a global "Master List".
Lab 8 Implementation: Dash enables inline passenger customization directly inside the booking stream. For multi-leg journeys, passengers can specify distinct preferences per leg (e.g., Lower Berth in 3A on Leg 1, Window Seat in CC on Leg 2) without losing context.
HCI Value: *Flexibility and Efficiency of Use (Nielsen #7)*.

> **Significant UX innovation over official IRCTC architecture: zero master-list roundtrips.**

8. Group Fare Transparency & Breakdown

IMPLEMENTED

Lab 7 Finding: P08 reported that aggregate totals obscured per-passenger costs, creating financial ambiguity.
Lab 8 Implementation: A transparent pricing breakdown that itemizes base fare per seat class, optional onboard catering (₹120/meal), and total group payable sum. Real-time updates reflect passenger additions or class modifications immediately.
HCI Value: *Visibility of System Status (Nielsen #1)*.

> **Detailed breakdown per traveler; dynamic recalculation without surprise checkout fees.**

9. Fallback & Recovery Journeys

IMPLEMENTED

Lab 7 Finding: P09 expressed anxiety over sudden Tatkal exhaustion leaving the traveler with zero contingency plan.
Lab 8 Implementation: Dash proactively computes backup options (e.g., 12016 Shatabdi, 12957 Rajdhani, 12985 Double Decker) with departure buffers, plus a one-click "Alert me if journey changes" intent toggle to mitigate distress.
HCI Value: *Help Users Recognize, Diagnose, & Recover from Errors (Nielsen #9)*.

> **Preserves group preferences and generates instant contingency alternatives.**

10. Ethical Boundary & Simulated Payment

IMPLEMENTED

Lab 7 Finding: P10 asked whether the app was executing financial transactions or generating authentic PNRs.
Lab 8 Implementation: Clear ethical boundary signposting. The final checkout step explicitly labels itself as a **Simulated Secure Payment Handoff**. A dedicated "Data & Trust" modal details exactly where prototype simulation ends.
HCI Value: *Ethical Interaction Design* and *Transparency*.

> **Prevents user deception; clearly delineates the academic prototype boundary.**

11. Information-Layered Train Cards

IMPLEMENTED

Lab 7 Finding: In Lab 6, train cards suffered from visual clutter: seat quotas, departure times, pantry badges, and days of run blurred together.
Lab 8 Implementation: Visual scannability redesign: large typography for departure/arrival times, clear transit duration badges, high-contrast class chips (1A, 2A, 3A, CC), and expandable detail panels.
HCI Value: *Aesthetic and Minimalist Design (Nielsen #8)*.

> **Critical decision data scannable in < 3 seconds; secondary info accessible on click.**

12. Theme Adjuster & WCAG AA Accessibility

IMPLEMENTED

Lab 7 Finding: P11 requested an accessible light theme for daylight viewing, and keyboard accessibility was previously unverified.
Lab 8 Implementation: Implemented dual-theme architecture (Dark default / Light mode) with CSS custom properties. Added visible keyboard focus rings (`focus-visible`), aria labels, and verified 4.5:1 text contrast ratios.
HCI Value: *Universal Usability & Accessibility (WCAG 2.1 AA)*.

> **Instant dark/light toggle; complete tab-navigation and focus indicator compliance.**

HCI AI511 | Lab 8: Improve Documentation — DASH Project
Page 4 of 10

1.3 Nielsen's 10 Usability Heuristics — Comprehensive Verification Matrix

To prove methodological rigor for the 4-mark implementation evaluation, the table below maps each of Jakob Nielsen's 10 Usability Heuristics to the exact state before and after the Lab 8 improvements:

Nielsen Heuristic	Lab 6 Baseline (Identified Defect)	Lab 8 Redesign (Implemented Resolution)
1. Visibility of System Status	Static seat numbers appeared live without indication of query time, source, or network failure.	Server-side live data status badges (`LIVE` vs `DEMO`), sync timestamps, and active stepper indicator.
2. Match Between System & Real World	Forced single Delhi station choice; ignored natural traveller mental models of Delhi as a destination city.	Delhi treated as an integrated station cluster (NDLS, DLI, DEE, DEC, SSB) matching passenger travel intent.
3. User Control & Freedom	Rigid recommendations pushed a single "best" train without explaining tradeoffs or allowing override.	User-controllable Smart Match priorities (Fare, Arrival, Comfort) and side-by-side trade-off inspection.
4. Consistency & Standards	Custom seat codes and ambiguous button actions differed across search and passenger entry screens.	Standardized railway class nomenclature (1A, 2A, 3A, CC, SL), standardized berth options, and consistent stepper navigation.
5. Error Prevention	Date was typed into a free-form input, causing invalid date crashes and missing format instructions.	Controlled 14-day interactive date chips and validated dropdown; impossible to enter an out-of-range date.
6. Recognition Rather Than Recall	Users had to memorize train schedules and stations when reviewing passenger information.	Persistent journey summary strip visible across all steps; saved passenger profiles pre-populated with IDs.
7. Flexibility & Efficiency of Use	Group booking forced editing a global master list, repeating the major interaction defect of IRCTC.	In-flow passenger preferences and multi-leg per-journey overrides; keyboard navigation enabled throughout.
8. Aesthetic & Minimalist Design	All features, live metrics, and filters crowded the main view, overwhelming users (P02).	Progressive disclosure: streamlined primary journey view; complex comparison tools open on explicit user intent.
9. Help Users Recognize/Recover	If a train had no seats, users had to start the entire search from scratch.	Fallback/Recovery Journey module displays immediate alternatives (e.g. Shatabdi / Rajdhani) with 1 click.
10. Help & Documentation	No explanation of how recommendations were derived or whether payment created a live booking.	Integrated "Data & Trust" modal, station guide explanations, and explicit simulated payment disclaimer.

HCI AI511 | Lab 8: Improve Documentation — DASH Project
Page 5 of 10

PART 2 GitHub Repository & Documentation (2 Marks)

The second evaluation criterion awards 2 marks for establishing an active GitHub repository, demonstrating professional version control discipline, and authoring comprehensive project documentation.

2.1 GitHub Repository Overview & Version Control Discipline

> **Active GitHub Repository URL:** `https://github.com/uttamshs/Railway-tatkal-ticket-booking-project`

Version Control Architecture GIT WORKFLOW

- **Git Remote:** Linked to origin on GitHub under team ownership.
- **Release Tag:** Semantic versioning maintained at `v3.0.0` (Dash Smart Journey Planner).
- **Commit Cadence:** Clean chronological commits recording initial builds, Lab 7 test instrumentation, and Lab 8 redesigns.
- **Secret Hygiene:** Strict `.gitignore` enforcement preventing check-in of API keys (`.env.local` containing RailRadar credentials ignored).

Team Structure & Contributions COLLABORATION

- **Harshit Kulshrestha:** Lead Developer & UI/UX Architect (Frontend implementation, Next.js components, Smart Match engine, Lab 7 data analysis).
- **Uttam (uttamshs):** Repository Maintainer & System Integrator (Git remote setup, backend API adapters, documentation, deployment pipeline).
- **Peer Pair Programming:** Joint heuristic reviews and usability testing debriefs.

2.2 Repository File Organization

The codebase adheres to standard Next.js 16 App Router architecture, ensuring strict modularity between presentation, business logic, and API endpoints:

```
dash/
├── app/
│   ├── api/
│   │   ├── availability/route.ts    # Upstream train class availability proxy
│   │   ├── live/route.ts            # Secure RailRadar live status endpoint (API key protected)
│   │   └── search/route.ts          # Multi-station cluster search proxy
│   ├── globals.css                  # Dual-theme tokens (dark/light), typography, focus states
│   ├── layout.tsx                   # Root HTML shell, responsive viewport metadata
│   └── page.tsx                     # Core 4-step interactive journey planner & state store
├── components/
│   └── Stepper.tsx                  # Accessible visual step progress indicator
├── docs/
│   ├── documentation.pdf            # This comprehensive Lab 8 submission report
│   ├── DASH_FREE_DEPLOYMENT_GUIDE.md# Production deployment walkthrough for Vercel
│   └── DASH_Report.md               # Summary feature release notes
├── lib/
│   ├── data.ts                      # Station cluster definitions, fallback demo timetables
│   ├── recommendation.ts            # Explainable Smart Match & trade-off ranking engine
│   └── types.ts                     # TypeScript interfaces (Train, Passenger, Berth, Priority)
├── public/                          # Optimized SVG/PNG assets and Dash branded icon
├── .env.example                     # Public template for railway API environment variables
├── .env.local                       # Local developer secrets (strictly git-ignored)
├── package.json                     # Project manifest (Next.js 16, React 19, TypeScript 5.7)
└── README.md                        # Primary project documentation & onboarding guide
```


HCI AI511 | Lab 8: Improve Documentation — DASH Project
Page 6 of 10

2.3 Technology Stack Specifications

Layer	Technology	Version	Design Rationale
Frontend Framework	Next.js (App Router)	16.3.7	Industry-standard React framework supporting server components and zero-bundle server logic.
UI Runtime	React & React-DOM	19.2.0	Modern concurrent rendering, hooks-based state management, high-performance UI diffing.
Type Safety	TypeScript	5.7.0	Strict interface contracts for railway APIs, group passenger data, and recommendation matrices.
Styling & Theming	Vanilla CSS Custom Properties	CSS3	Zero external UI bloat; instantaneous CSS variable switching for accessible Dark / Light themes.
Railway Data APIs	RailRadar API & IR Community API	REST	Live schedule retrieval, platform status, and seat availability with server-side caching.
Testing & Verification	Playwright Headless Suite	Latest	Automated browser testing, responsive layout verification, and publication PDF rendering.

2.4 Production Deployment Architecture (Moving Away from Localhost)

In response to feedback that the prototype should be accessible on public infrastructure rather than confined to `localhost`, Dash is architected for zero-cost deployment on **Vercel**:

> **Vercel Cloud Deployment Pipeline:**
>
> 1. Connect GitHub repository `uttamshs/Railway-tatkal-ticket-booking-project` to Vercel dashboard.
> 2. Configure root directory to `dash/`.
> 3. Inject Production Environment Variables in Vercel settings: `RAIL_API_BASE_URL=https://indian-railway-api.onrender.com`, `RAILRADAR_BASE_URL=https://api.railradar.in`, and private `RAILRADAR_API_KEY`.
> 4. Deploy via automatic CI/CD: Produces a globally available HTTPS production endpoint for evaluators.

HCI AI511 | Lab 8: Improve Documentation — DASH Project
Page 7 of 10

PART 3 Viva Voce Defense & Evaluation Preparation (2 Marks)

To ensure full marks in the Viva evaluation (conducted by Course Faculty and Teaching Assistant Abhinav), this section prepares direct, articulate, evidence-grounded answers to the critical technical and HCI questions covering the **Lab 6 build**, **Lab 7 testing**, and **Lab 8 improvements**.

VIVA Q1 What was the central problem with your Lab 6 build, and what did Lab 7 user testing reveal?

Authoritative Answer:
"Our Lab 6 build proved that we could program a functional railway booking flow, but Lab 7 proved that a functional interface is not necessarily a usable one. In Lab 7, we tested 11 real participants (yielding an average SUS score of 74.09). We discovered that the hardest part of train booking is not typing text into fields—it is *decision-making under uncertainty*.
Users struggled with three primary barriers:

- **Terminal Ambiguity:** Users travelling to Delhi had no idea whether to select NDLS, DLI, DEE, DEC, or SSB (P03 quote: *'I don't know which Delhi station I should select'*).
- **Cognitive Load & Opaque Recommendations:** The interface was cluttered (P02 quote: *'Too many things competing for attention'*), and our recommendation badge didn't explain trade-offs (P05).
- **Group Coordination Friction:** Real booking involves families and colleagues with conflicting needs (seniors needing lower berths, children with adults, and per-leg preferences), which our Lab 6 build completely ignored."

VIVA Q2 What is the specific innovation in Dash that solves a major design defect in the official IRCTC portal?

Authoritative Answer:
"In the official IRCTC portal, if a traveller books a connecting journey or multiple legs—for example, Leg 1 in 3AC where they want a Lower Berth, and Leg 2 in Chair Car (CC) where they want a Window Seat—IRCTC does not allow them to customize preferences directly inside the booking flow. The user is forced to break their workflow, navigate to a separate global 'Master List' page, edit their profile, and restart their booking.
In Dash, we solved this by implementing **In-Flow Passenger Customization with Per-Leg Overrides**. Within the passenger step, users can dynamically assign and override class and berth preferences for each leg independently without losing their active group state. This embodies Nielsen's heuristic of *Flexibility and Efficiency of Use*."

VIVA Q3 How does your 'Delhi Station Cluster Intelligence' work from an HCI and algorithmic standpoint?

Authoritative Answer:
"Traditional railway systems force users to choose a specific station code before seeing trains. However, user mental models think in terms of cities, not 4-letter station codes. Delhi is a metropolitan region served by multiple major terminals (New Delhi [NDLS], Old Delhi [DLI], Sarai Rohilla [DEE], Delhi Cantt [DEC], and Shakur Basti [SSB]).
Dash introduces 'Delhi — All Stations' as an aggregated cluster. Our server-side route queries all terminals concurrently and reconciles the results. We then provide contextual recommendations—for example, recommending Delhi Sarai Rohilla (DEE) for morning express trains because of faster transit times and proximity to North/West Delhi, while explaining the exact rationale. This satisfies Nielsen's *Match Between System and the Real World*."

HCI AI511 | Lab 8: Improve Documentation — DASH Project
Page 8 of 10

VIVA Q4 **Why did you implement both Live API integration and a Demo Fallback state? How does this relate to ethical design?**

Authoritative Answer:
"During Lab 7, participant P06 explicitly stated: *'I can't tell whether the availability is actually real. If it's dummy numbers mention it, otherwise show real time availability.'*
Free public railway APIs are notoriously subject to rate limiting and downtime. Many student prototypes either crash during evaluations or fabricate fake numbers to appear live. We took an ethical, engineering-driven approach:
1. We built real server-side proxy routes interfacing with RailRadar and Indian Railway API endpoints.
2. If the API succeeds, the UI displays a green `LIVE` badge with an exact refresh timestamp.
3. If the external API times out or hits quotas, Dash gracefully falls back to a curated demo timetable and *visibly labels it as DEMO FALLBACK*.
This maintains 100% demonstration reliability while respecting Nielsen's *Visibility of System Status* and upholding ethical design by never deceiving the user."

VIVA Q5 **How did your team handle Version Control, Git branching, and collaboration for Lab 8?**

Authoritative Answer:
"Our repository is hosted publicly at `https://github.com/uttamshs/Railway-tatkal-ticket-booking-project`.
We utilized a structured version control workflow:
• **Feature Isolation & Commits:** Commit history reflects discrete milestones: baseline setup, removal of redundant nested submodules, and feature-complete integration.
• **Security Best Practices:** We created `.env.example` with dummy templates, and strictly excluded `.env.local` containing private API keys (RailRadar) in `.gitignore`.
• **Semantic Release:** The improved build is versioned as `v3.0.0`.
• **Team Division:** Harshit led the interactive UI architecture, Progressive Disclosure components, and Lab 7 empirical data synthesis; Uttam managed git repository synchronization, backend proxy routes, and deployment documentation."

VIVA Q6 **What is the purpose of the 'Simulated Payment Handoff' screen?**

Authoritative Answer:
"Participant P10 in Lab 7 raised concerns about whether clicking 'Book' would initiate a bank transfer or generate an authentic IRCTC PNR. In HCI and ethical interface design, establishing the *Prototype Boundary* is critical.
We explicitly label the checkout CTA as 'Proceed to Simulated Payment' and include a prominent notice clarifying that Dash is an academic HCI prototype that does not process monetary transactions or issue live government tickets. This avoids the 'dark pattern' of false affordances."

HCI AI511 | Lab 8: Improve Documentation — DASH Project
Page 9 of 10

2.5 Roadmap to Final Project Evaluation (14 Oct – 31 Oct 2026)

Following the completion of Lab 8 and Lab 9 (14 Oct 2026), our team will utilize the remaining two-week refinement sprint before the final evaluation on **31 October 2026** to execute the following planned enhancements:

Timeline Phase	Milestone Objectives	Deliverable
Phase 1 (14–20 Oct)	Incorporate final Lab 8 viva feedback; refine multi-leg seat map visualization; complete PNR tracking screen.	Build v3.1.0 Release
Phase 2 (21–27 Oct)	Conduct secondary SUS validation benchmark; stress test serverless API proxies; deploy staging environment on Vercel.	Public Staging Live URL
Phase 3 (28–31 Oct)	Prepare interactive demonstration script; package final report artifact; present final project evaluation.	Final Project Showcase (31 Oct)

HCI AI511 | Lab 8: Improve Documentation — DASH Project Page 10 of 10